

2025.08.25

UART기반 디지털 시계 (STOPWATCH-WATCH)

[HARMAN] Semicon Academy 2기

발표자 : 이윤지

CONTENTS

1 Introduction

2 Block Diagram

3 ASM & Module

4 Simulation 결과

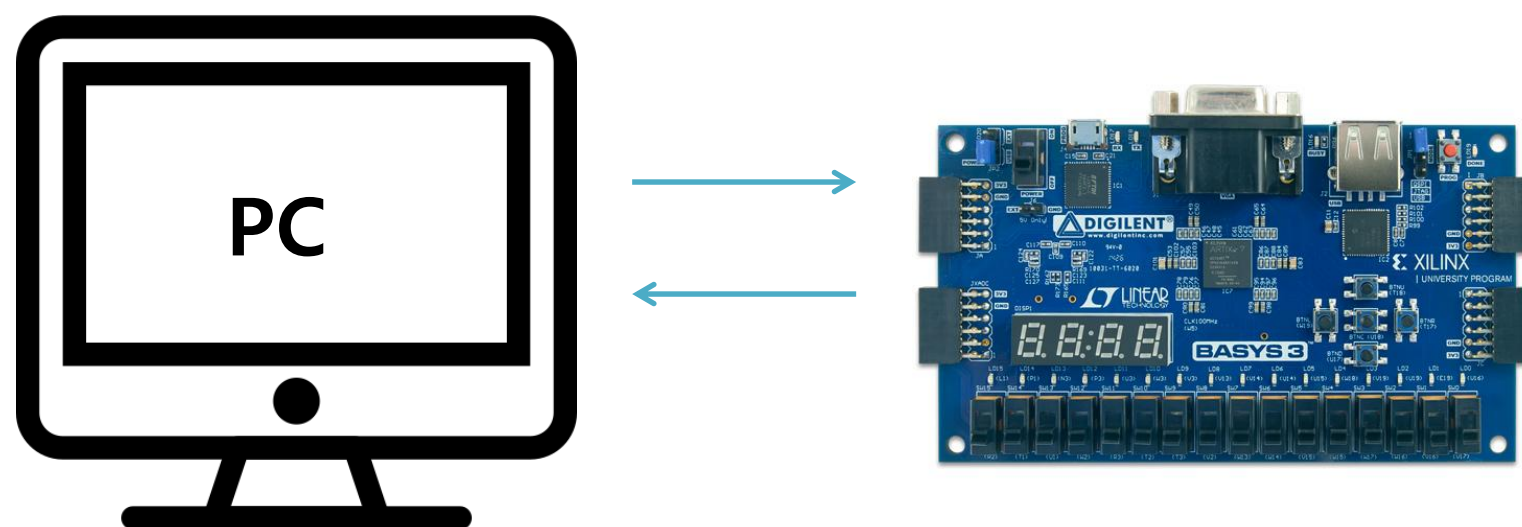
5 동작 영상

6 Trouble shooting

01

Introduction

프로젝트 목표

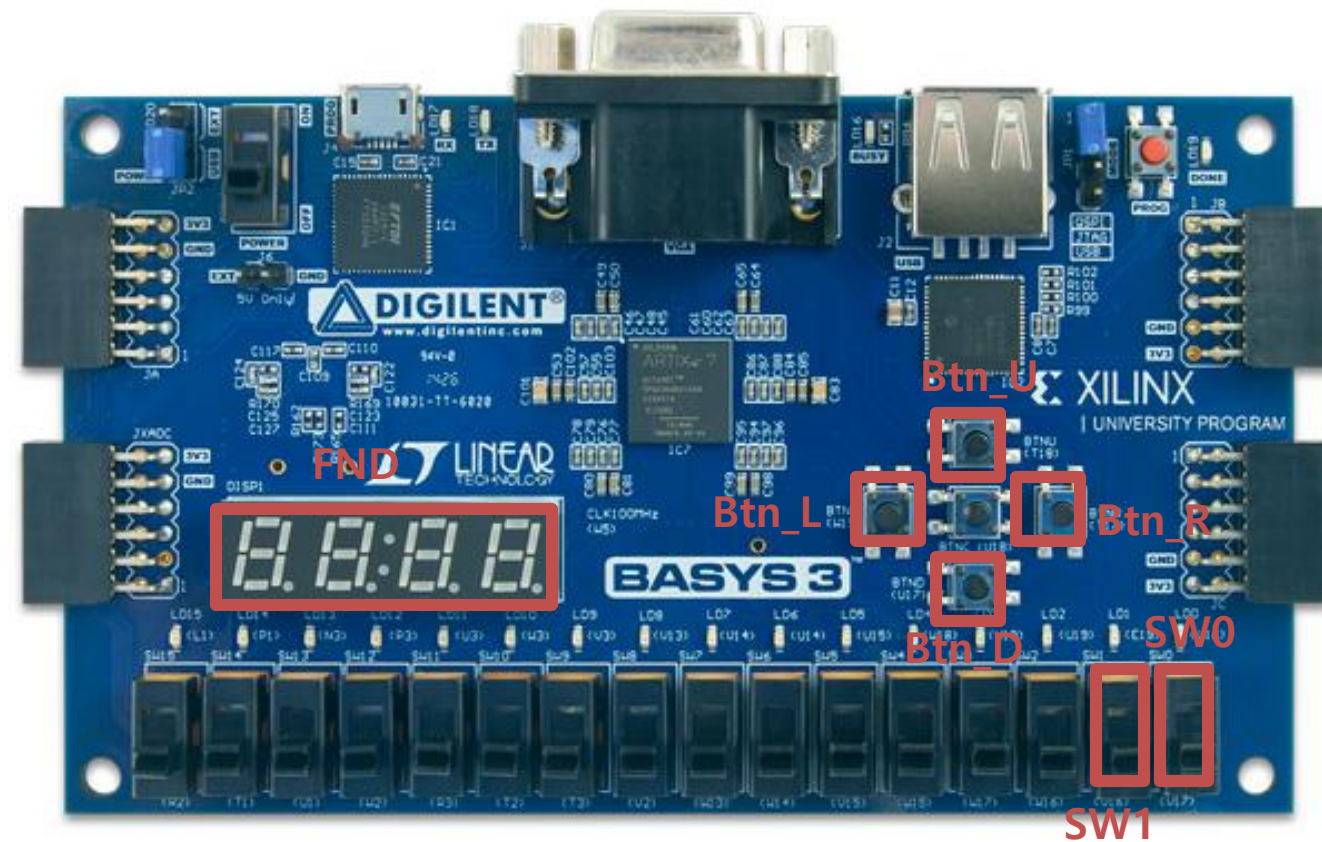


- 기존에 설계한 stopwatch-watch를
uart의 입력으로 제어할 수 있는 디지털 시계
- SW와 BTN 뿐만 아니라 UART(PC) 로 동일 기능
- PC 의 우선 순위 보장 : PC 모드 해제 후 SW
복귀

01

Introduction

Basys3 보드

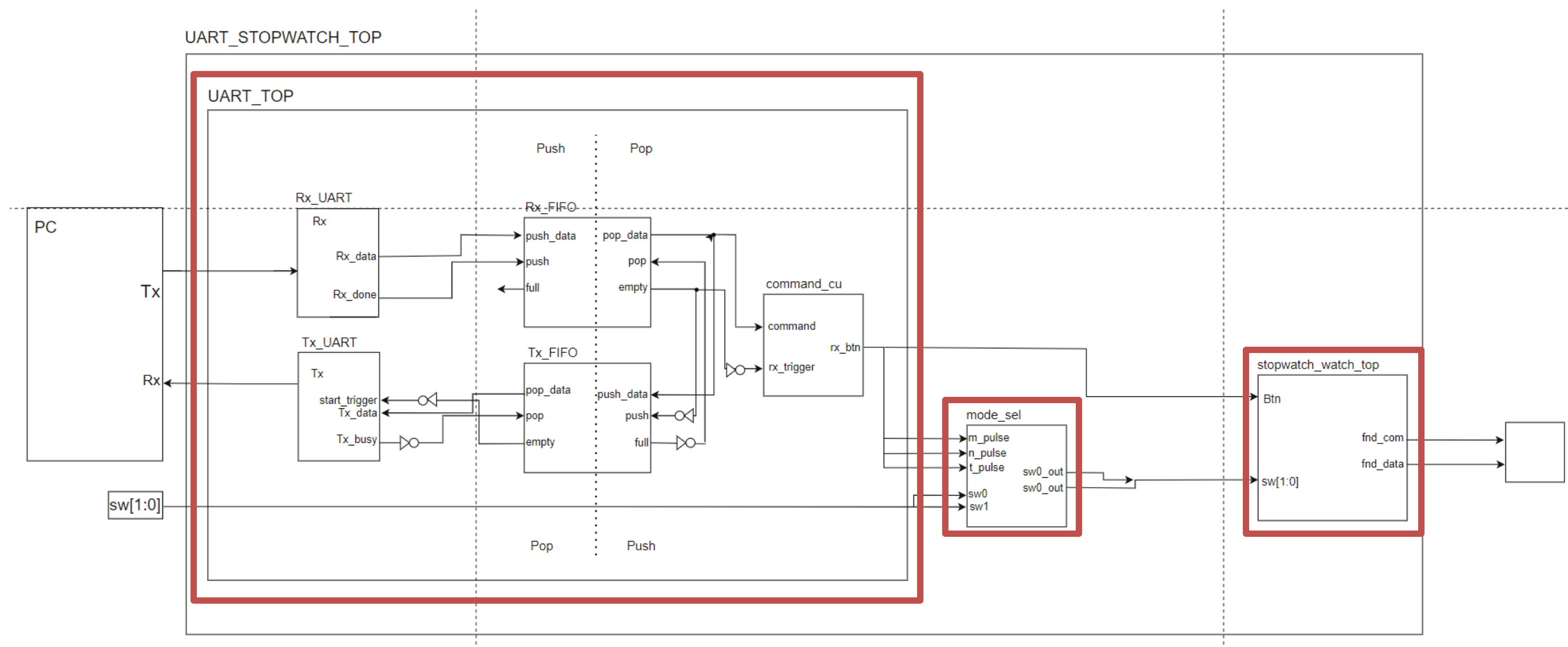


- Xilinx Vivado
- VerilogHDL
- 입출력: SW 2개,
BTN5개 (위/아래/좌/우/센터),
7-Segment

02

Block Diagram

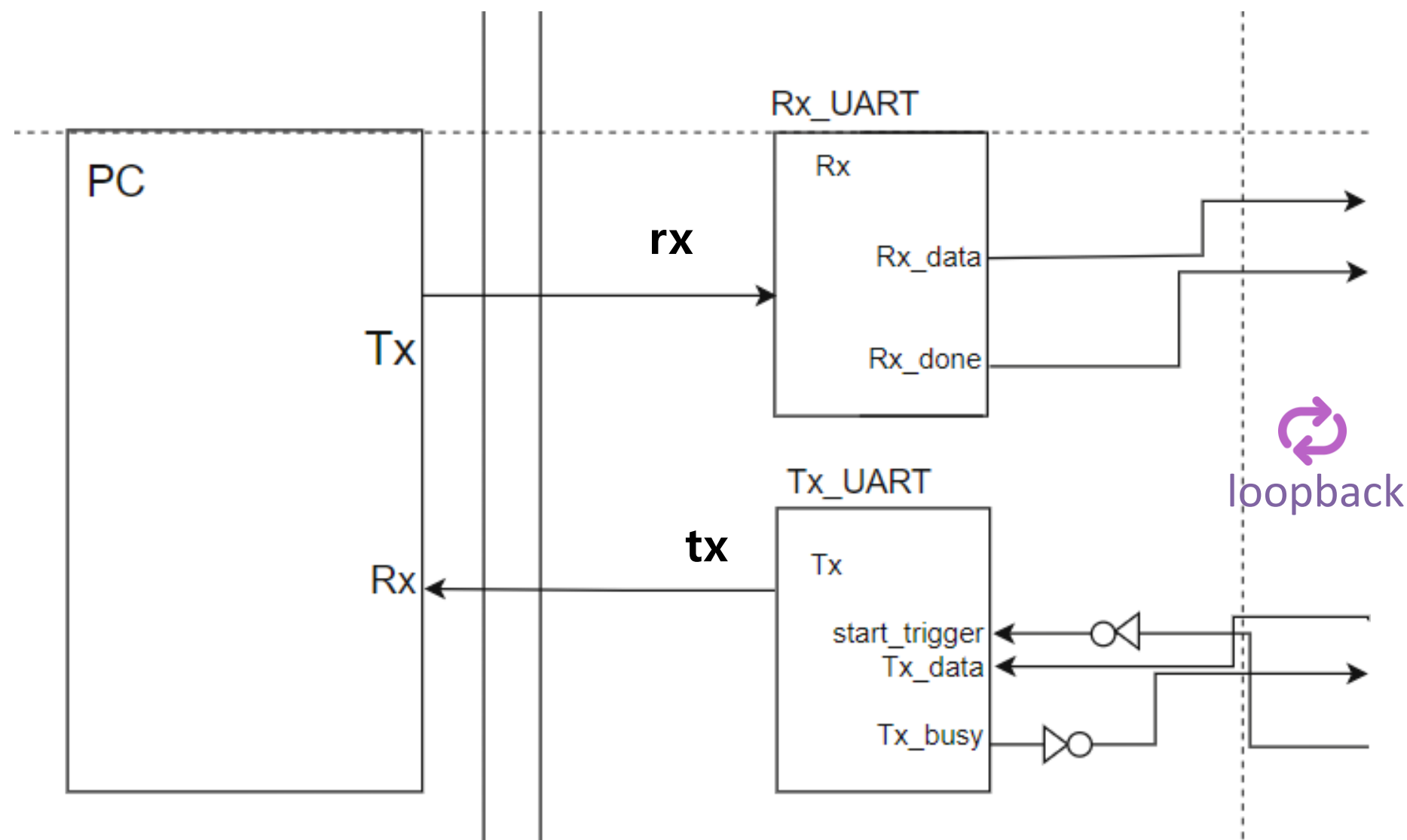
전체 Block Diagram



02

Block Diagram

UART Block Diagram

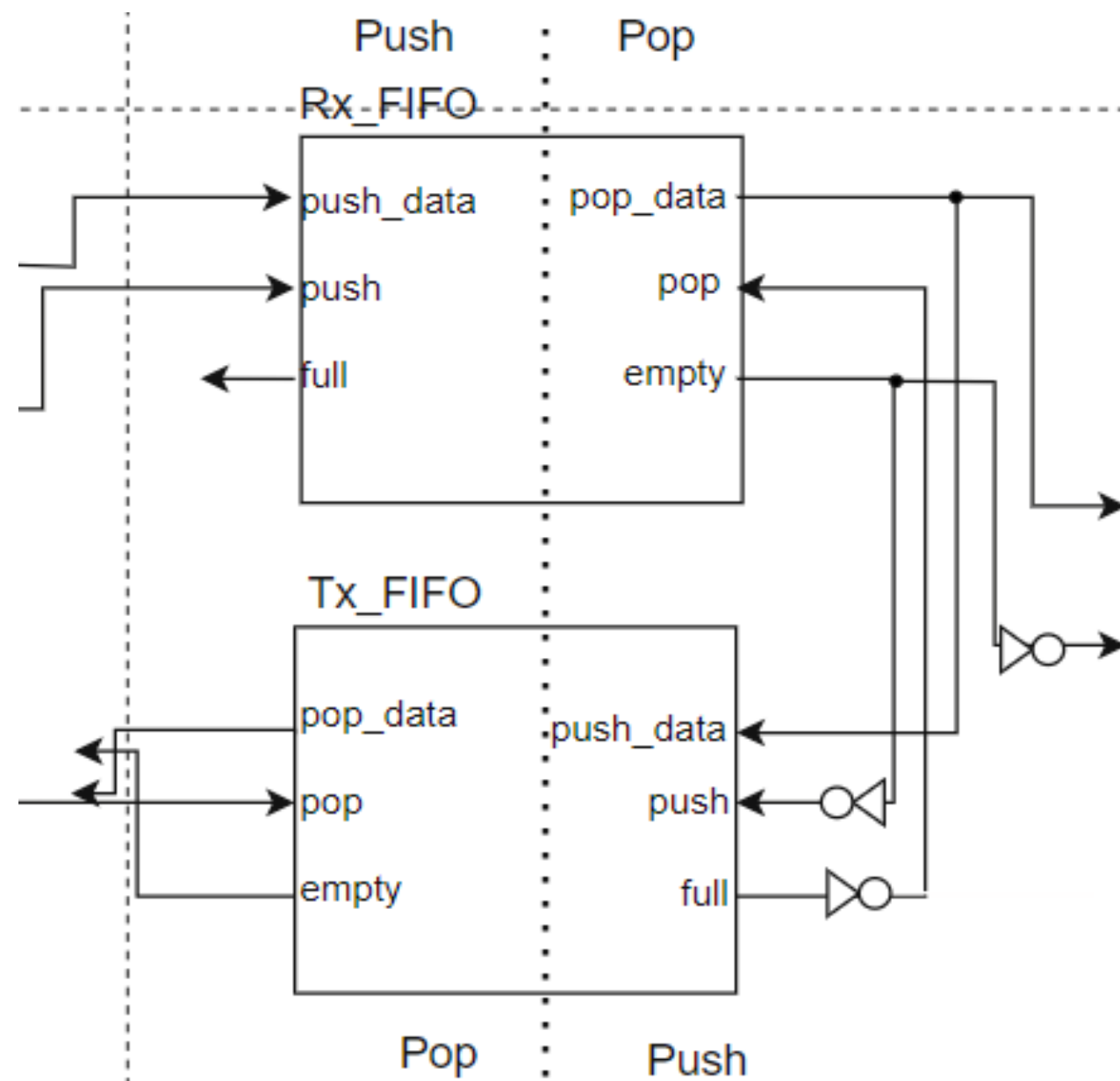


- baud tick gen, uart_rx, uart_tx로 구성
- baud tick gen에서 출력된 b_tick이
uart_rx, tx 로 들어감
- uart_rx : 입력 rx가 rx_data로 출력 >
uart_tx : tx_data로 입력
- uart_tx의 start_trigger = 1 >
tx로 출력 = loopback 구조

02

Block Diagram

FIFO Block Diagram

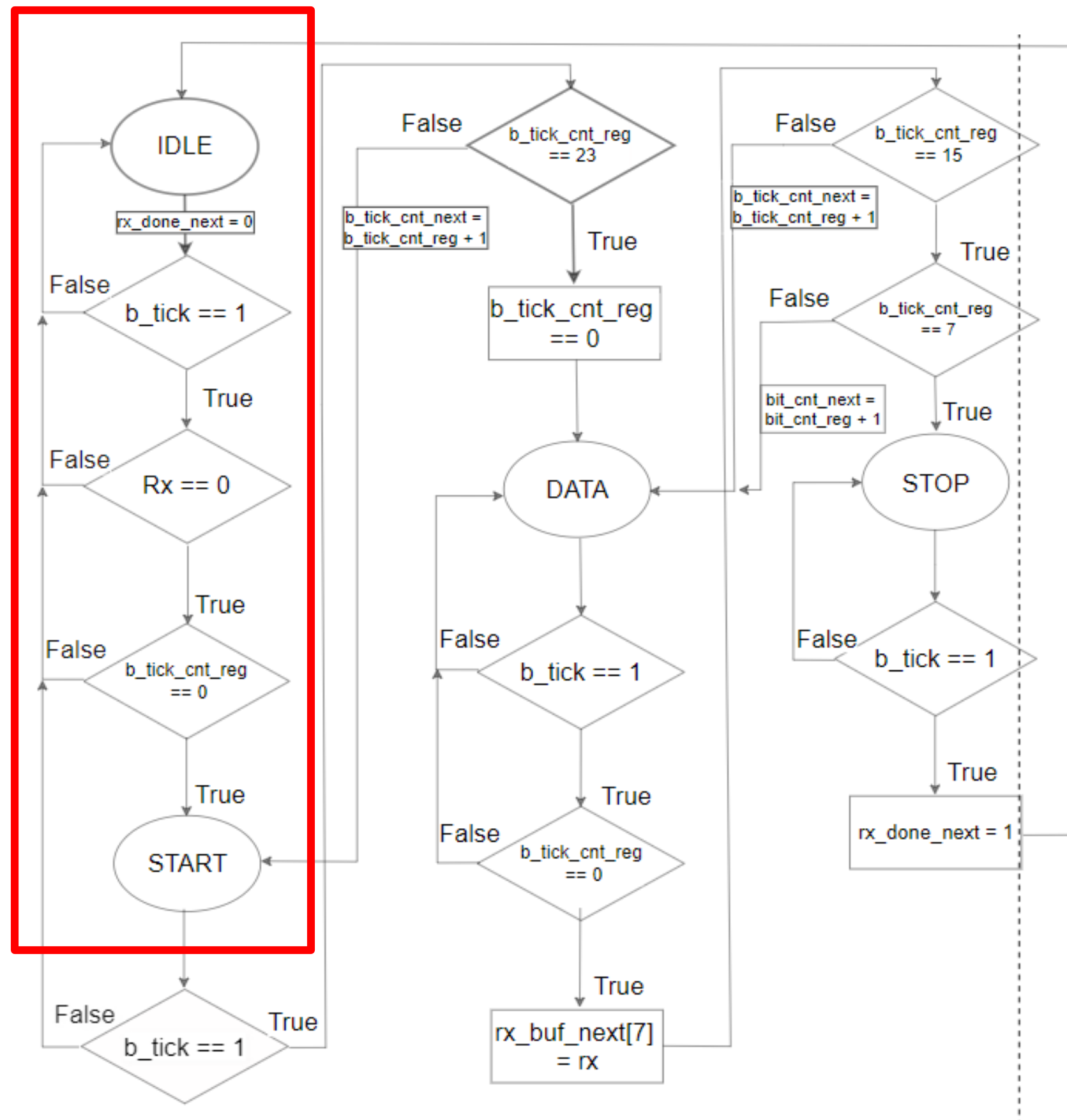


- 8비트의 데이터를 저장하고 읽는 4칸의 큐
- register file : 데이터를 보관,
fifo control unit : 읽기, 쓰기, 상태 (full/empty) 관리
- 쓰기(push) > 가득 차지 않았을 때 현재 쓰기 위치(wptr) 데이터를 저장
- 읽기(pop) > 현재 읽기 위치(rpctr)의 값 pop_data로 출력

03

ASM & Module

ASM(RX) Chart :
IDLE -> START



03

ASM & Module

IDLE -> START

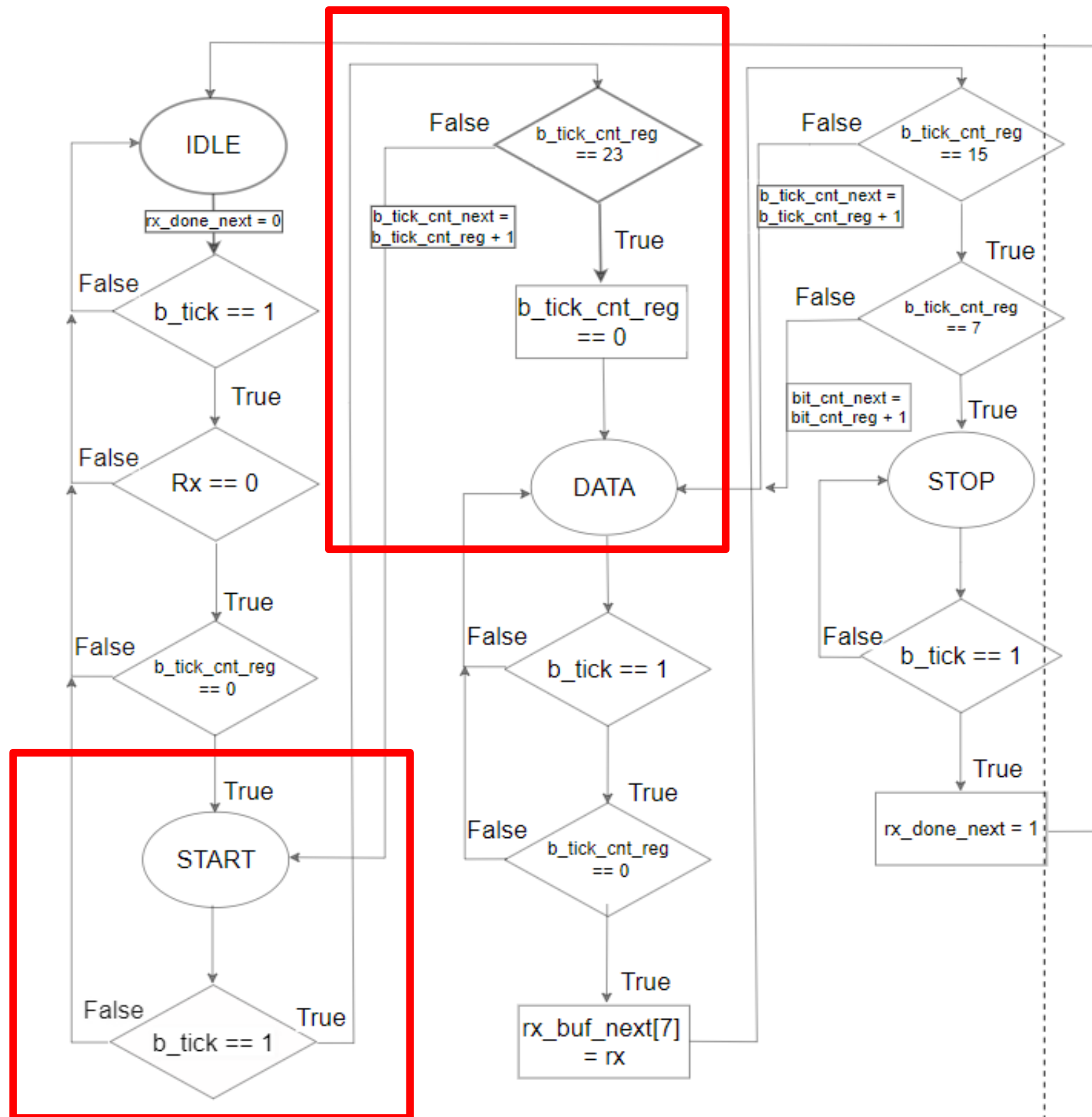
```
//next CL
always @(*) begin
    next = state;
    b_tick_cnt_next = b_tick_cnt_reg;
    bit_cnt_next = bit_cnt_reg;
    rx_done_next = rx_done_reg;
    rx_buf_next = rx_buf_reg;
    case (state)
        IDLE: begin
            rx_done_next = 1'b0;
            if (b_tick) begin
                if (rx == 0) begin
                    b_tick_cnt_next = 0;
                    next = START;
                end
            end
        end
    end
end
```

- $b_tick = 1$ > 상태 전이 판단
- $rx_done_reg = 0$ 으로 초기화 유지
- $rx == 0$ (start 비트 하강) 시 $b_tick_cnt_reg = 0$ 다음 상태 준비

03

ASM & Module

ASM(RX) Chart :
START -> DATA



03

ASM & Module

START -> DATA

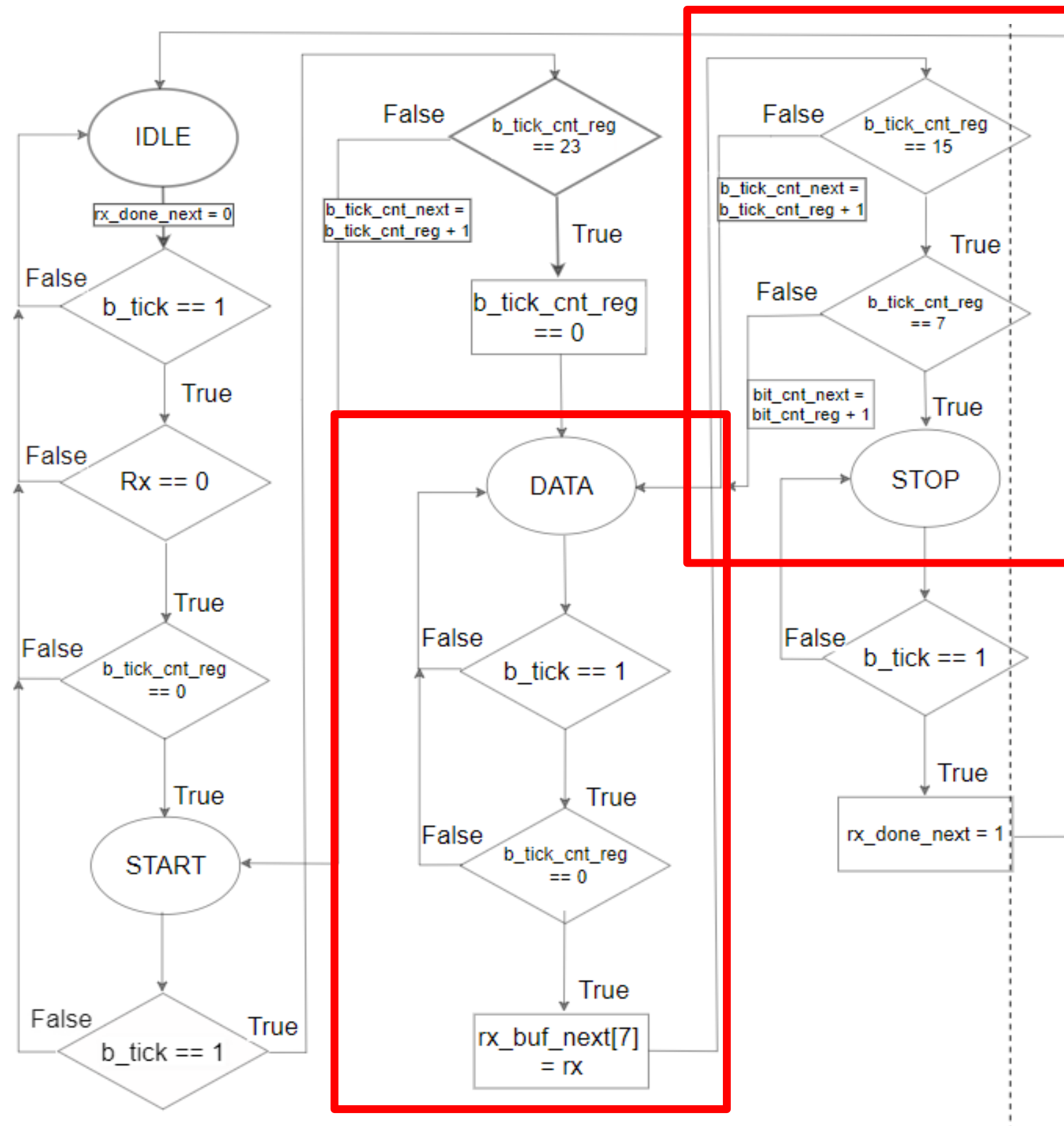
```
START: begin
  if (b_tick) begin
    if (b_tick_cnt_reg == 23) begin
      bit_cnt_next = 0;
      b_tick_cnt_next = 0;
      next = DATA;
    end else begin
      b_tick_cnt_next = b_tick_cnt_reg + 1;
    end
  end
end
```

- 한 비트를 16개 tick -> 중앙에서 샘플링
- b_tick을 24번 카운트
- bit_cnt 리셋 후 DATA 로 진입
- 그 외에 bit_cnt가 24까지 도달하지 못하면 b_tick마다 카운터 + 1

03

ASM & Module

ASM(RX) Chart :
DATA -> STOP



03

ASM & Module

DATA -> STOP

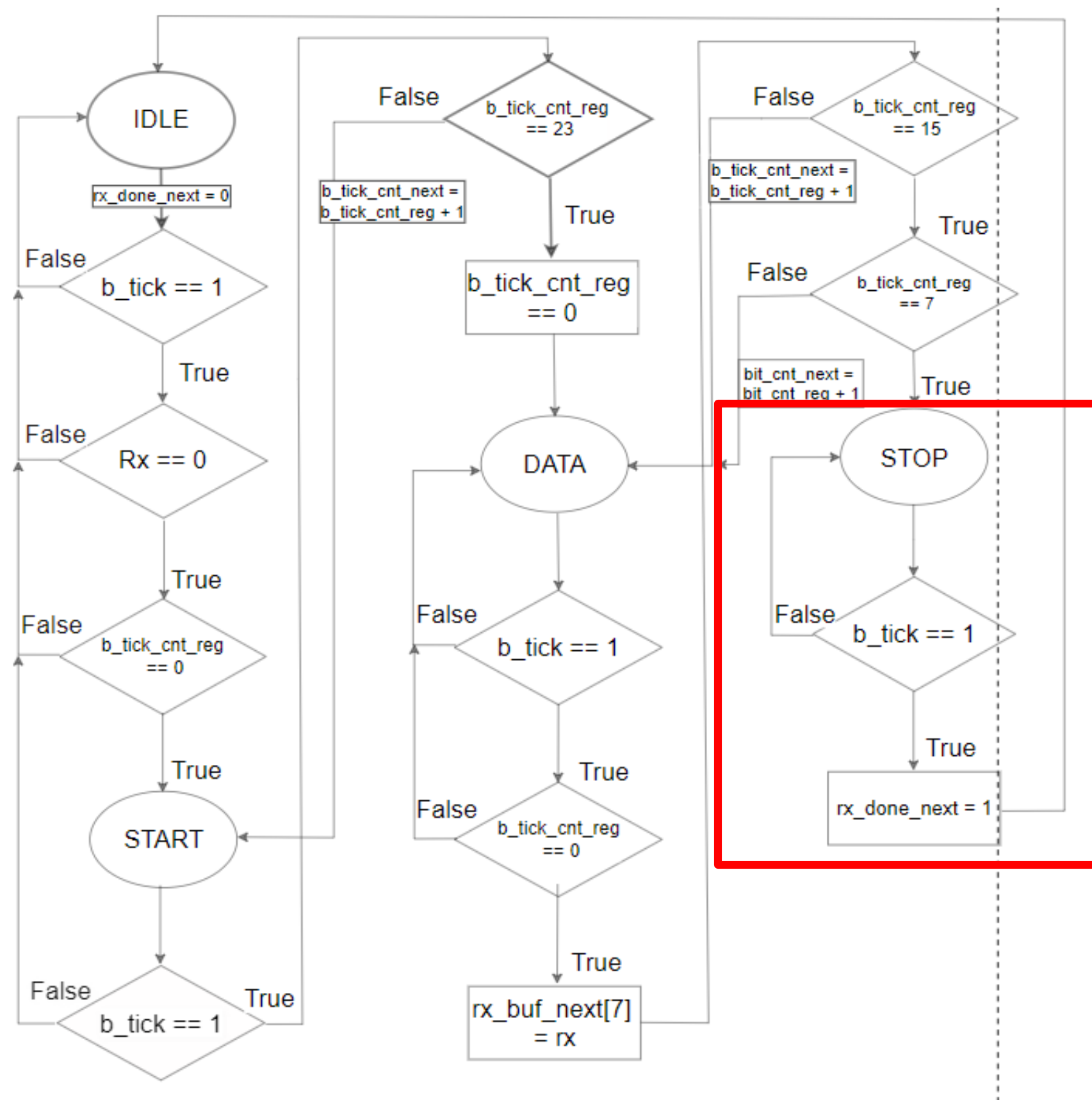
```
DATA: begin
  if (b_tick) begin
    if (b_tick_cnt_reg == 0) begin
      rx_buf_next[7] = rx;
    end
    if (b_tick_cnt_reg == 15) begin
      if (bit_cnt_reg == 7) begin
        next = STOP;
      end else begin
        b_tick_cnt_next = 0;
        bit_cnt_next = bit_cnt_reg + 1;
        rx_buf_next = rx_buf_reg >> 1;
      end
    end else begin
      b_tick_cnt_next = b_tick_cnt_reg + 1;
    end
  end
end
```

- 매 비트 시작 (bit_cnt = 0)에 중앙 샘플
- bit_cnt가 15에서 한 비트 종료
- 기존의 rx_data를 오른쪽으로 한칸 shift -> 새 rx의 값은 가장 왼쪽 비트(MSB)에 넣음
- 결국 LSB 부터 MSB까지 순서대로 8bit 누적

03

ASM & Module

ASM(RX) Chart :
STOP -> IDLE



03

ASM & Module

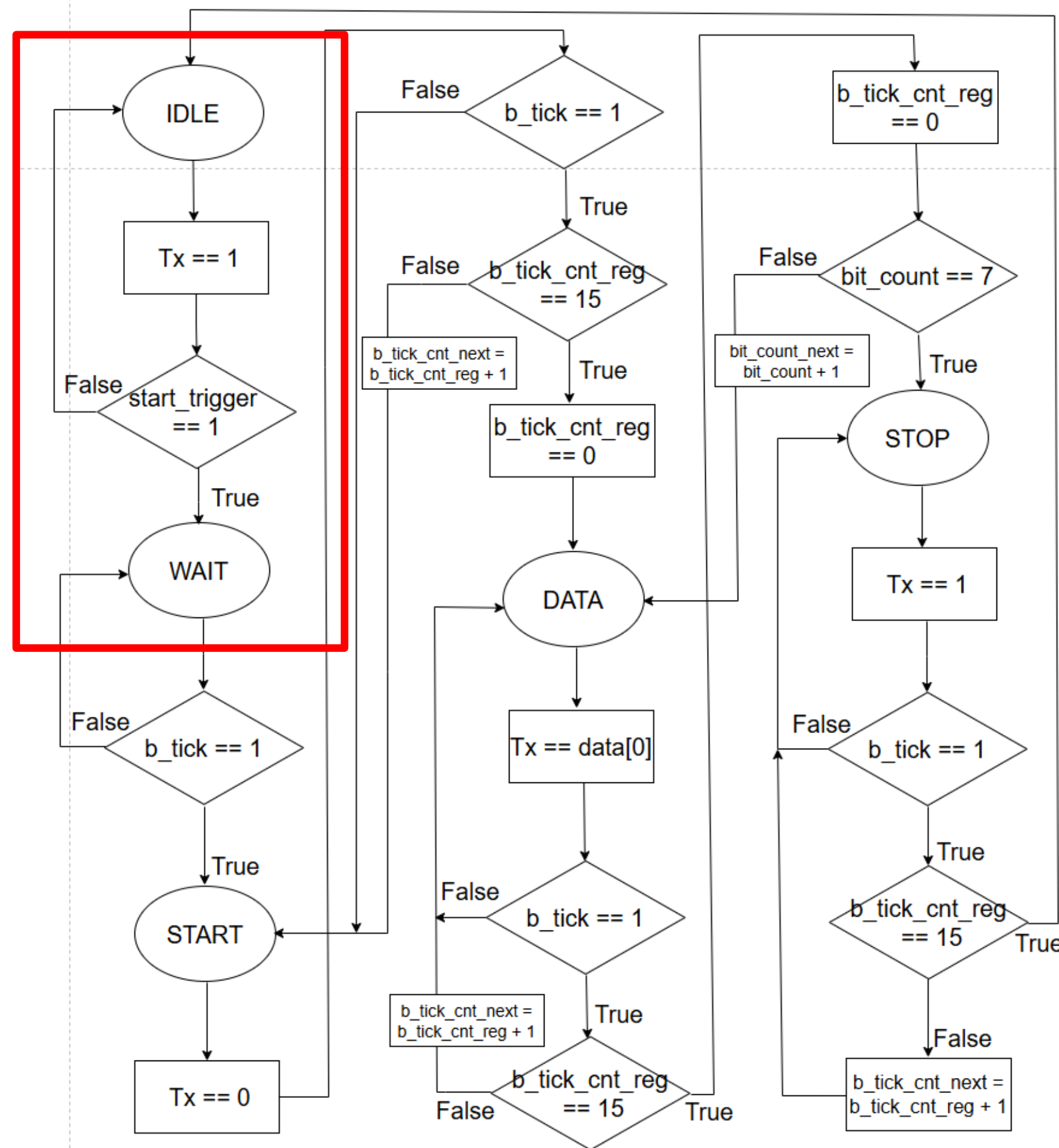
STOP -> IDLE

```
STOP: begin
  if (b_tick) begin
    rx done next = 1'b1;
    next = IDLE;
  end
end
```

- B_tick 들어오면 rx_done = 1
- Rx_done = 1 이 되는 순간 rx_data를 Rx_fifo로 push
- IDLE 로 복귀

ASM & Module

ASM(TX) Chart : IDLE -> WAIT



03

ASM & Module

IDLE -> WAIT

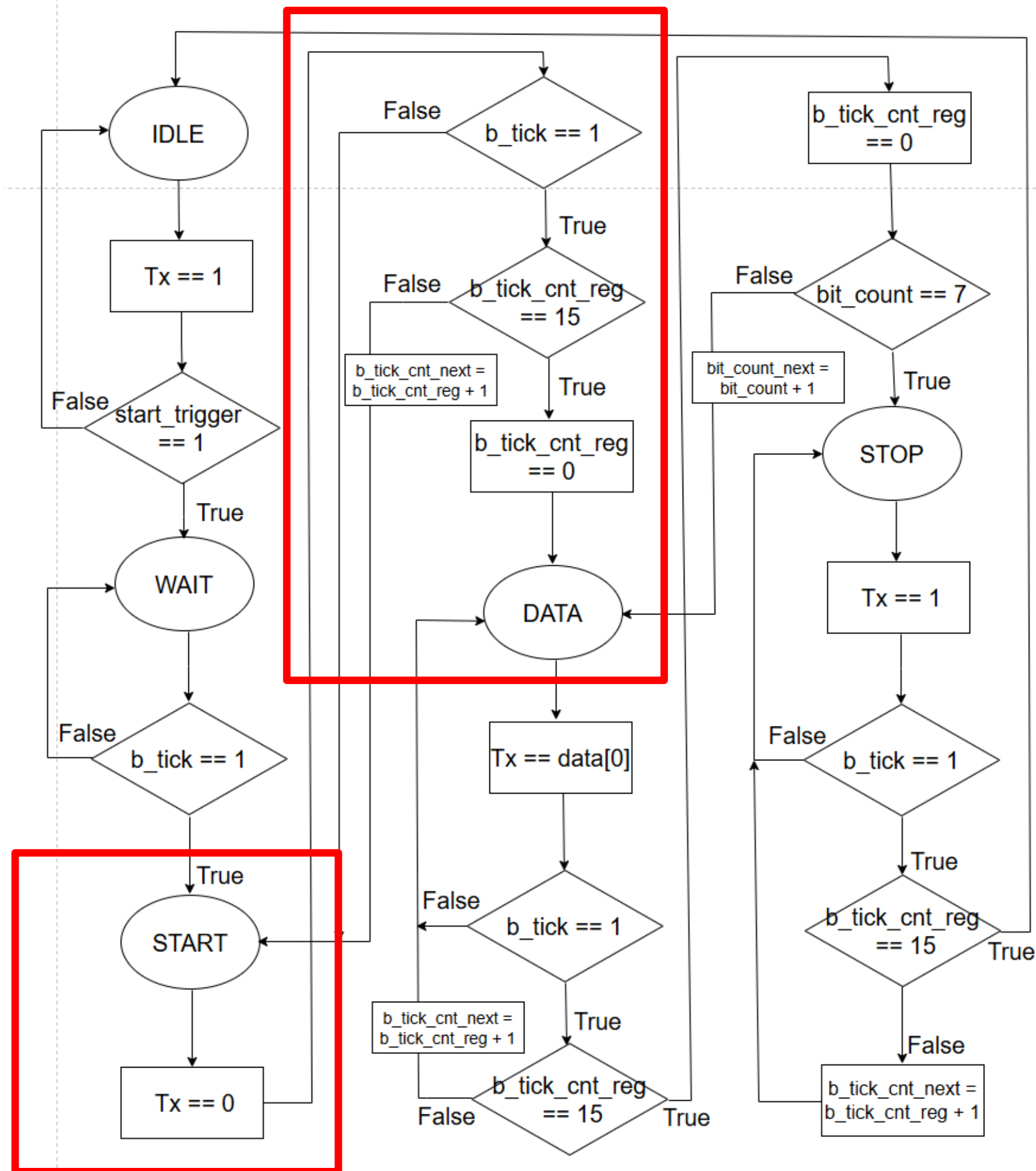
```
IDLE: begin
    tx_next = 1'b1;
    tx_busy_next = 1'b0;
    if (start trigger) begin
        tx_busy_next = 1'b1;
        data_next = tx_data;
        next = WAIT;
    end
end
```

- Start_trigger = 1 -> 상위 로직에서 받아온 tx_data를 data_reg에 래치
- Tx_busy = 1을 통해 새 데이터는 받지 않도록 표시함
- WAIT로 진입

03

ASM & Module

ASM(TX) Chart : WAIT -> START



03

ASM & Module

WAIT -> START

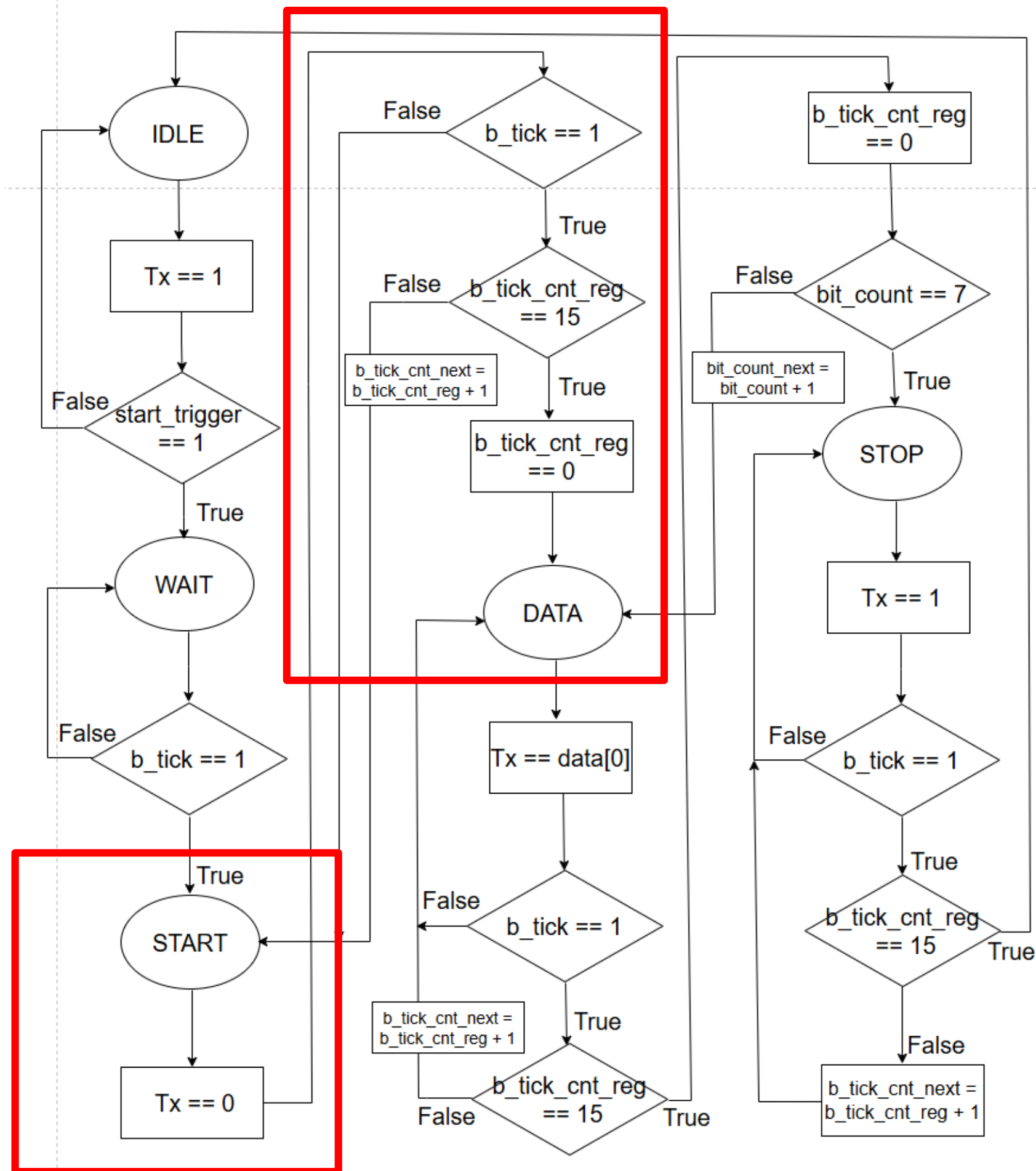
```
WAIT: begin
    if (b_tick) begin
        b_tick_cnt_next = 0;
        next = START;
    end
end
```

- B_tick이 들어오면 시작
- Start_trigger 뒤에 다음 b_tick까지 잠깐 대기
- B_tick이 들어오는 순간 bit_cnt= 0으로 초기화
- START 로 진입

03

ASM & Module

ASM(TX) Chart :
START -> DATA



03

ASM & Module

START -> DATA

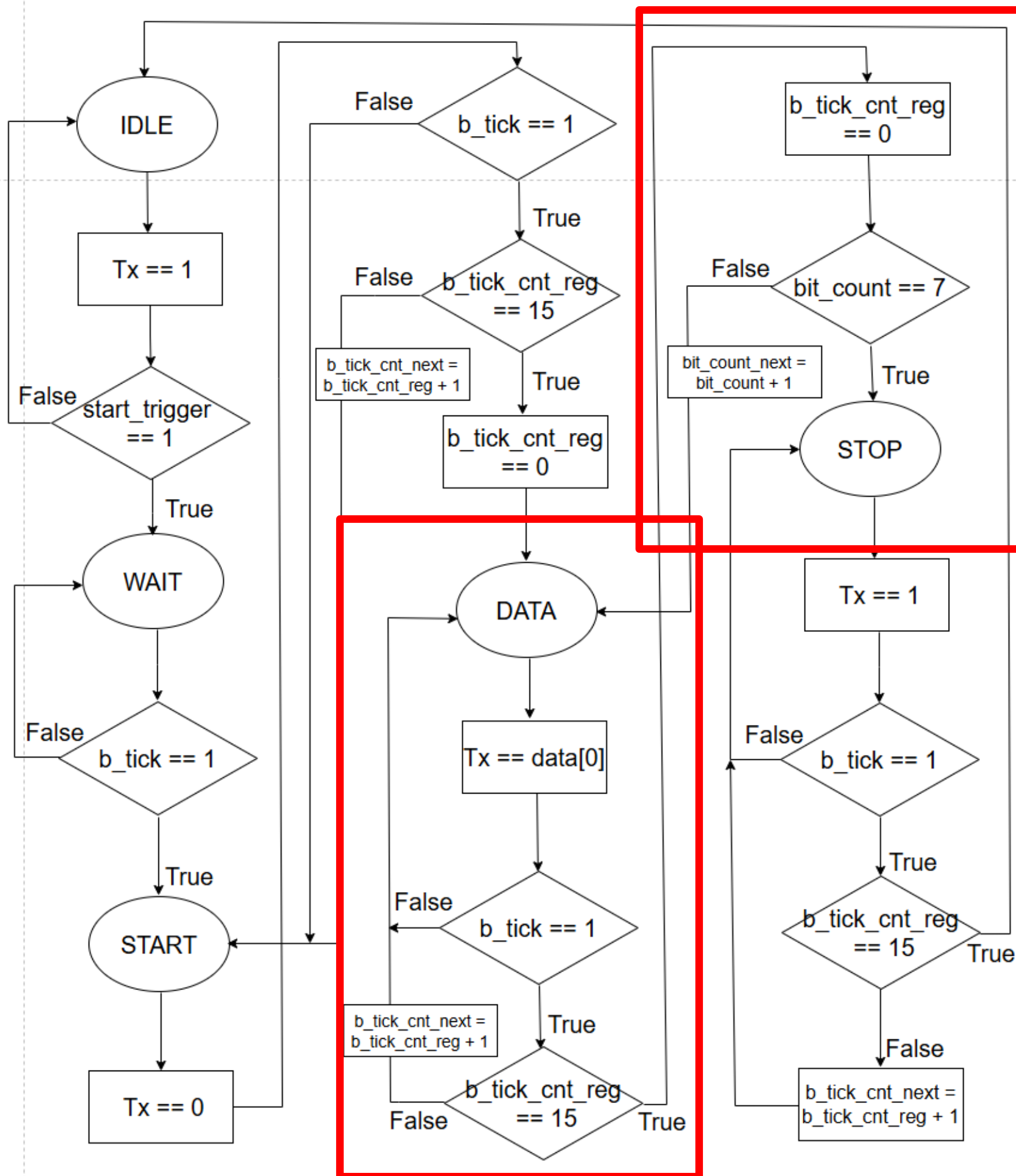
```
START: begin
    //output tx
    tx_next = 1'b0;
    if (b_tick) begin
        if (b_tick_cnt_reg == 15) begin
            b_tick_cnt_next = 0;
            bit_count_next = 0; //reset
            next = DATA;
        end else begin
            b_tick_cnt_next = b_tick_cnt_reg + 1;
        end
    end
end
end
```

- Tx_reg = 0 을 16tick동안 유지
- B_tick = 15가 되는 순간 bit_cnt = 0
- 그 외에는 bit_cnt를 매순간 1씩 카운트
- DATA로 진입

03

ASM & Module

ASM(TX) Chart :
DATA -> STOP



03

ASM & Module

DATA -> STOP

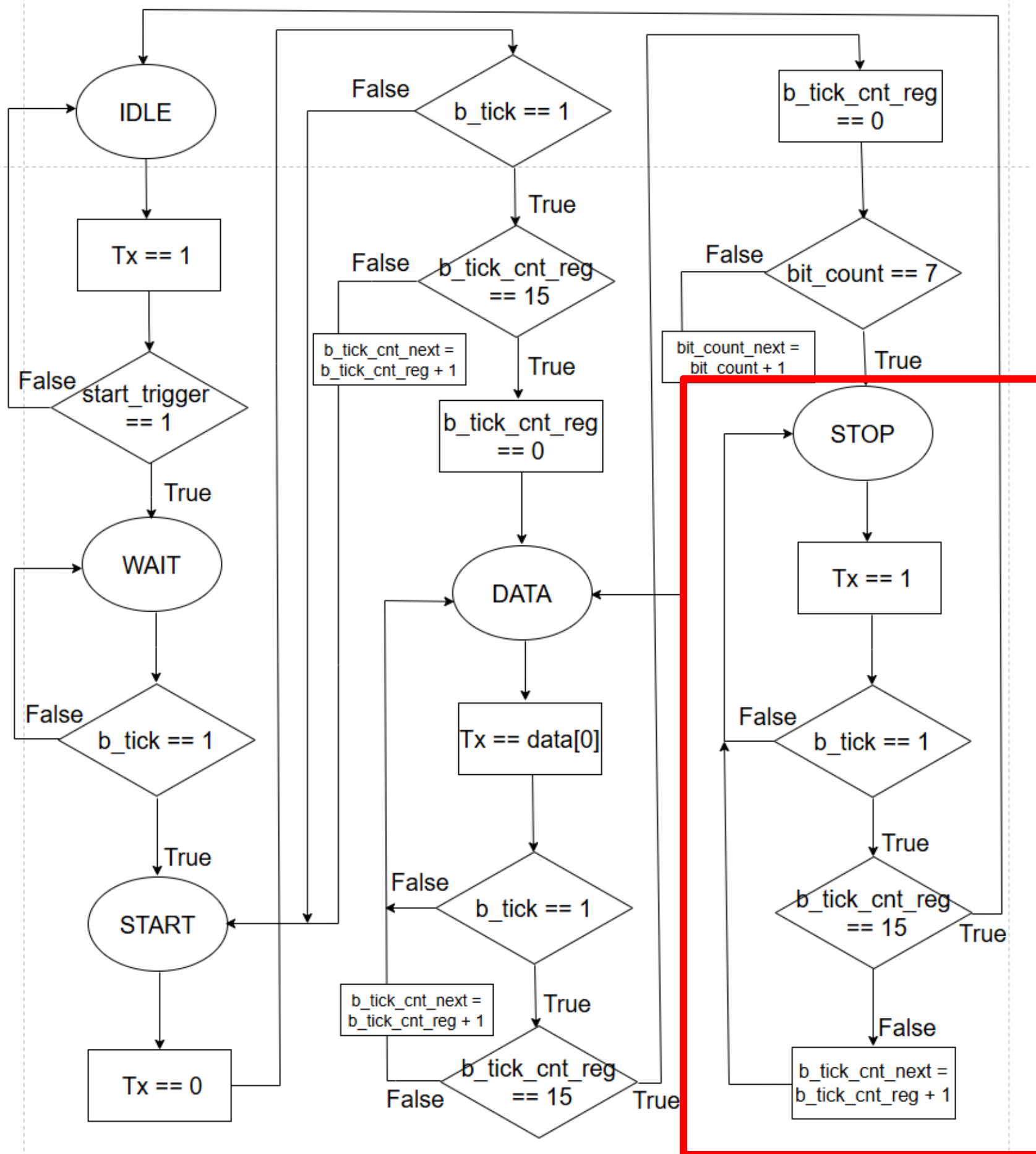
```
DATA: begin
  tx_next = data_reg[0];
  if (b_tick) begin
    if (b_tick_cnt_reg == 15) begin
      b_tick_cnt_next = 0;
      if (bit_count == 7) begin
        bit_count_next = 0;
        next = STOP;
      end else begin
        bit_count_next = bit_count + 1;
        data_next = data_reg >> 1;
      end
    end else begin
      b_tick_cnt_next = b_tick_cnt_reg + 1;
    end
  end
end
```

- LSB부터 8bit 전송
- 1bit 전송에 16tick 유지
- 16tick 마다 data_reg를 오른쪽으로 shift
- 마지막 비트의 16tick 완료 시 STOP로 진입

03

ASM & Module

ASM(TX) Chart :
STOP -> IDLE



03

ASM & Module

STOP -> IDLE

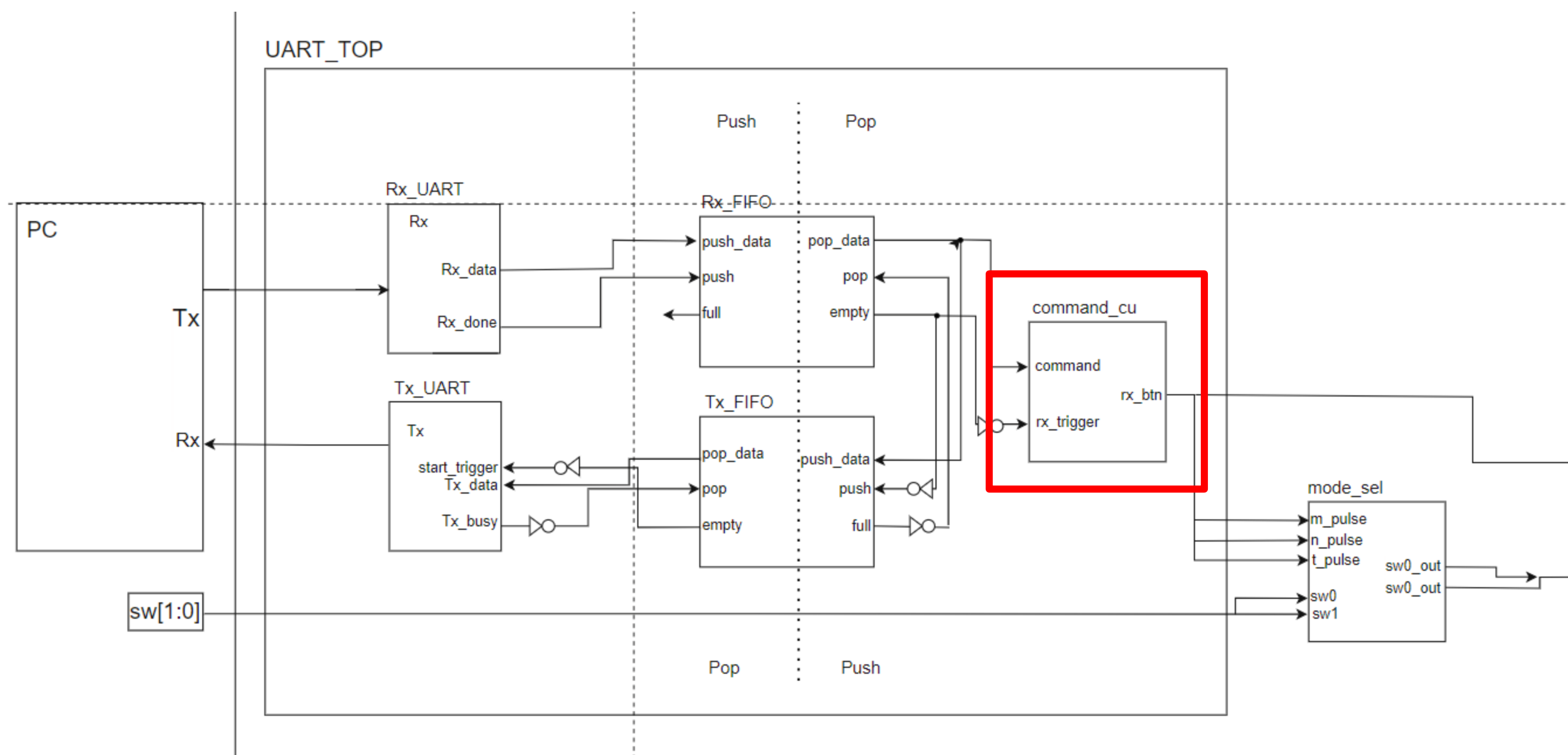
```
STOP: begin
    tx_next = 1'b1;
    if (b_tick) begin
        if (b_tick_cnt_reg == 15) begin
            tx_busy_next = 1'b0;
            next = IDLE;
        end else begin
            b_tick_cnt_next = b_tick_cnt_reg + 1;
        end
    end
end
end
```

- Tx_reg = 1을 16tick 유지
- IDLE : Tx_busy =1 -> Tx_busy =0 비트 전송이 끝남을 알려줌
- IDLE로 복귀

03

Block Diagram

COMMAND_CONTROL_UNIT



03

Block Diagram

COMMAND_CONTROL_UNIT

```
localparam [7:0] r = 8'h72, s = 8'h73, c = 8'h63, S = 8'h53, M = 8'h4D, H = 8'h48;
localparam [7:0] m = 8'h6D, t = 8'h74;
localparam [7:0] n = 8'h30; //n = 0
```

ASCII 명령코드

```
always @(*) begin

    btn_r_next = 1'b0;
    btn_s_next = 1'b0;
    btn_c_next = 1'b0;
    btn_S_next = 1'b0;
    btn_M_next = 1'b0;
    btn_H_next = 1'b0;
    btn_m_next = 1'b0;
    btn_t_next = 1'b0;
    btn_n_next = 1'b0;

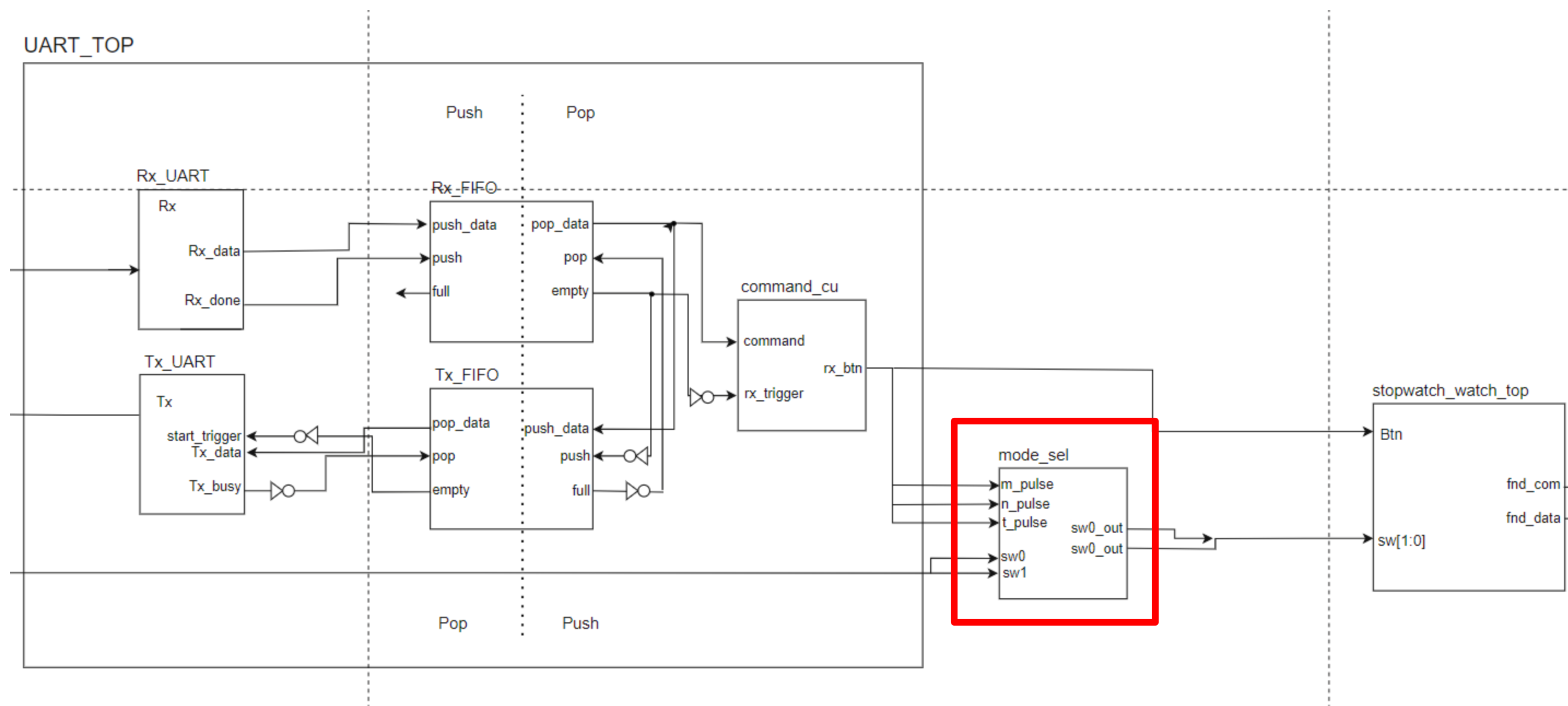
    //start_trig_next = 1'b0;
    if (rx_trigger) begin
        case (command)
            r : btn_r_next = 1'b1;
            s : btn_s_next = 1'b1;
            c : btn_c_next = 1'b1;
            S : btn_S_next = 1'b1;
            M : btn_M_next = 1'b1;
            H : btn_H_next = 1'b1;
            m : btn_m_next = 1'b1;
            t : btn_t_next = 1'b1;
            n : btn_n_next = 1'b1;
        endcase
    end
end
```

- PC가 uart로 ASCII 문자를 보냄
- Command_cu가 명령 코드로 디코딩해 동작함

03

Block Diagram

MODE_SEL



03

Block Diagram

MODE_SEL

```
case (state)
  S_IDLE: begin // 기본 상태
    sw1_next = sw1; // mode = 보드 스위치 그대로
    sw0_next = sw0; // time = 보드 스위치 그대로
    if (m_edge || t_edge) begin ←
      // PC 모드로 넘어가기 직전: 보드 상태를 기반으로 sw가 눌렸다면 반대로 설정
      // pc 모드로 가기 전 준비
      pc_mode_ff_n = sw1 ^ (m_edge ? 1'b1 : 1'b0); // m,t 가 들어왔을 때 (xor 연산) 1'b1 출력
      pc_time_ff_n = sw0 ^ (t_edge ? 1'b1 : 1'b0); // sw ^ 1'b1 = ~sw, sw ^ 1'b0 = sw
      state_next = S_PC;
      // 즉, 스위치 올려도 무시하게 됨
    end
  end
end
```

- S_IDLE : 보드 값 그대로 표시
- m,t 입력 : sw0, sw1
들어오면 sw값 토글(xor)
- sw0, sw1의 값을 반대로 함
- Sw값 무시하고 S_PC 진입

03

Block Diagram

MODE_SEL

```

S_PC: begin // (보드 스위치 무시)
    if (n_edge) begin // n(0) = 해제 요청
        // 모드는 즉시 STOPWATCH로 강제 표시
        sw1_next = IDLE_MODE; // sw1 = 기본으로
        sw0_next = sw0; // sw0은 그대로 반영
        pc_mode_ff_n = IDLE_MODE;
        state_next = S_HOLD;
    end else begin // m,t가 계속 오면 PC 내부 값 반대로 → 그 값을 실시간 출력으로 사용
        pc_mode_ff_n = pc_mode_ff ^ (m_edge ? 1'b1 : 1'b0); // pc모드에서 m,t가 계속 들어오면
        pc_time_ff_n = pc_time_ff ^ (t_edge ? 1'b1 : 1'b0); // pc내부 값을 계속 반대로 해서 모드 바뀌게 해줌
        sw1_next = pc_mode_ff_n; // 그 값 실시간 출력
        sw0_next = pc_time_ff_n;
    end
end
end

```

- S_PC : 보드 SW 무시
- m,t 입력 : 들어올 때마다 xor 토글 해서 즉시 반영
- Pc의 m,t 계속 들어와도 모드가 계속 바뀜
- n_edge : sw1 = IDLE, sw0 = sw0 적용 후 S_HOLD 진입

03

Block Diagram

MODE_SEL

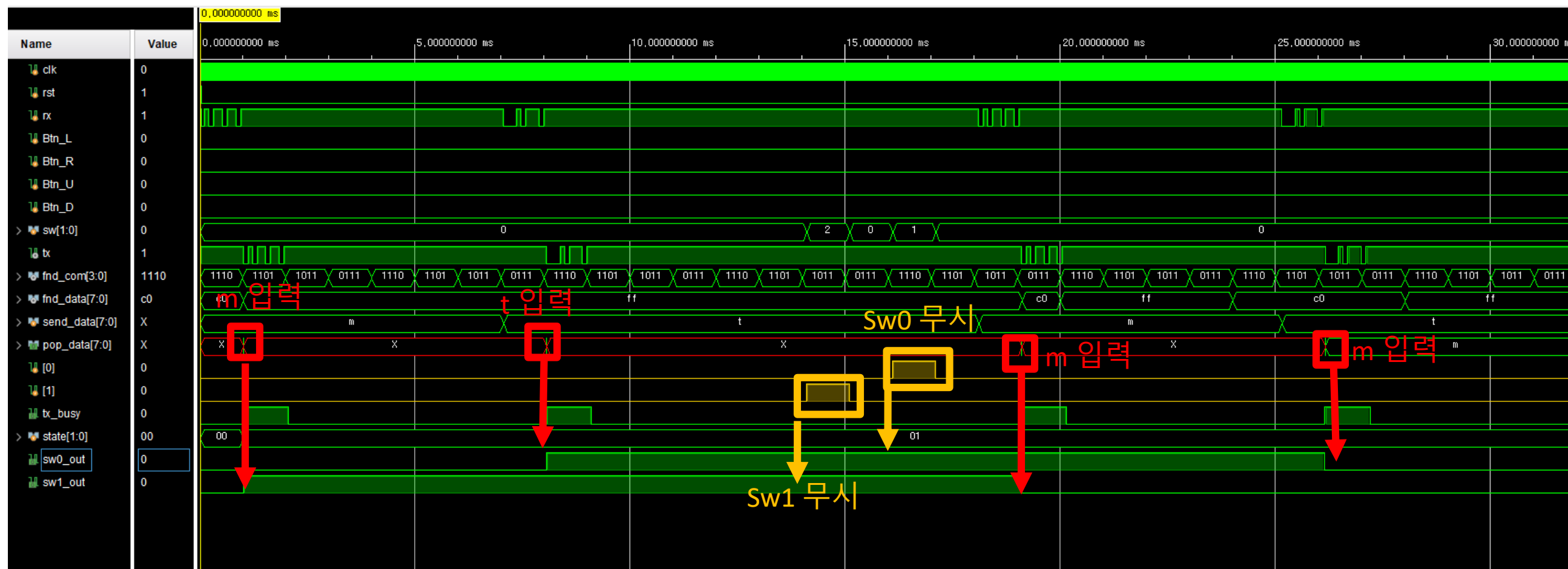
```
// n(0) 직후 잠깐 머무르는 상태
// HOLD : STOPWATCH, sw1 change -> IDLE
S_HOLD: begin
    sw1 next = IDLE MODE; //모드 강제 고정: STOPWATCH
    sw0 next = sw0; // 보드 sw0의 동작은 그대로 적용
    if (sw1_change) // 보드 sw1이 이전 값에서 '변하면' > IDLE 복귀
        state_next = S_IDLE;
end
```

- 모드 고정 : sw1 = IDLE_mode (stopwatch 유지)
- sw0 = sw0 그대로 반영 -> 즉시 표시
- 복귀 조건 : sw1의 변화
- S_IDLE로 복귀

04

Simulation 결과

MODE_SEL



```
//pc 모드 > 스위치 무시

send_data = "m";
send_uart(send_data);
#(5 * MS);

send_data = "t";
send_uart(send_data);
#(5 * MS);

sw[1] = 1'b1;
#(1 * MS);

sw[1] = 1'b0;
#(1 * MS);

sw[0] = 1'b1;
#(1 * MS);

sw[0] = 1'b0;
#(1 * MS);

send_data = "m";
send_uart(send_data);
#(5 * MS);

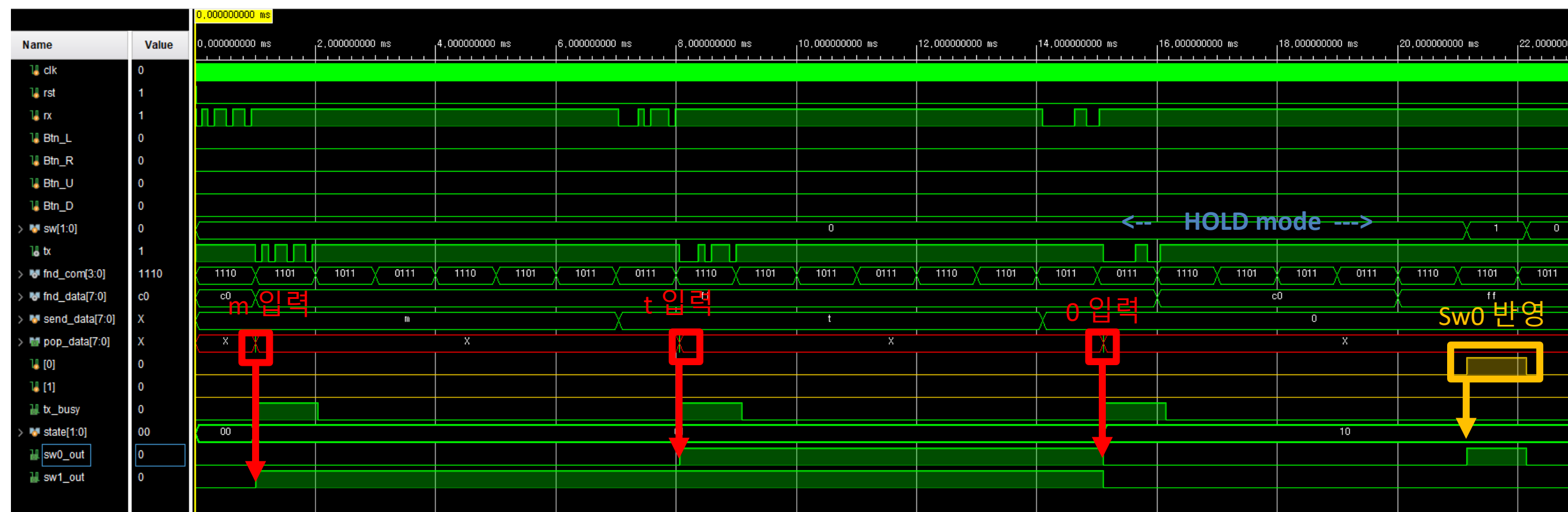
send_data = "t";
send_uart(send_data);
#(5 * MS);
```

- Pc -> m, t 입력 -> Pc모드로 진입(sw 무시) -> sw[1] 입력 -> sw[0] 입력 -> pc -> m, t 입력

04

Simulation 결과

MODE_SEL : sw0



// PC 모드 진입 → Watch-HH:MM 로 전환 → 보드 스위치 무시 확인
// '0' 수신 → HOLD 진입 → HOLD에서 sw0는 반영됨 확인
// (주의) 우리 RTL은 HOLD탈출이 sw1_change로만 됨. sw0만 바뀌어도 유지.

```
Btn_R = 1'b1;
#105_000; // 100us
Btn_R = 1'b0;
```

```
send_data = "m";
send_uart(send_data);
#(5 * MS);
```

```
send_data = "t";
send_uart(send_data);
#(5 * MS);
```

```
send_data = "0";
send_uart(send_data);
#(5 * MS);
```

```
sw[0] = 1'b1;
#(1 * MS);
```

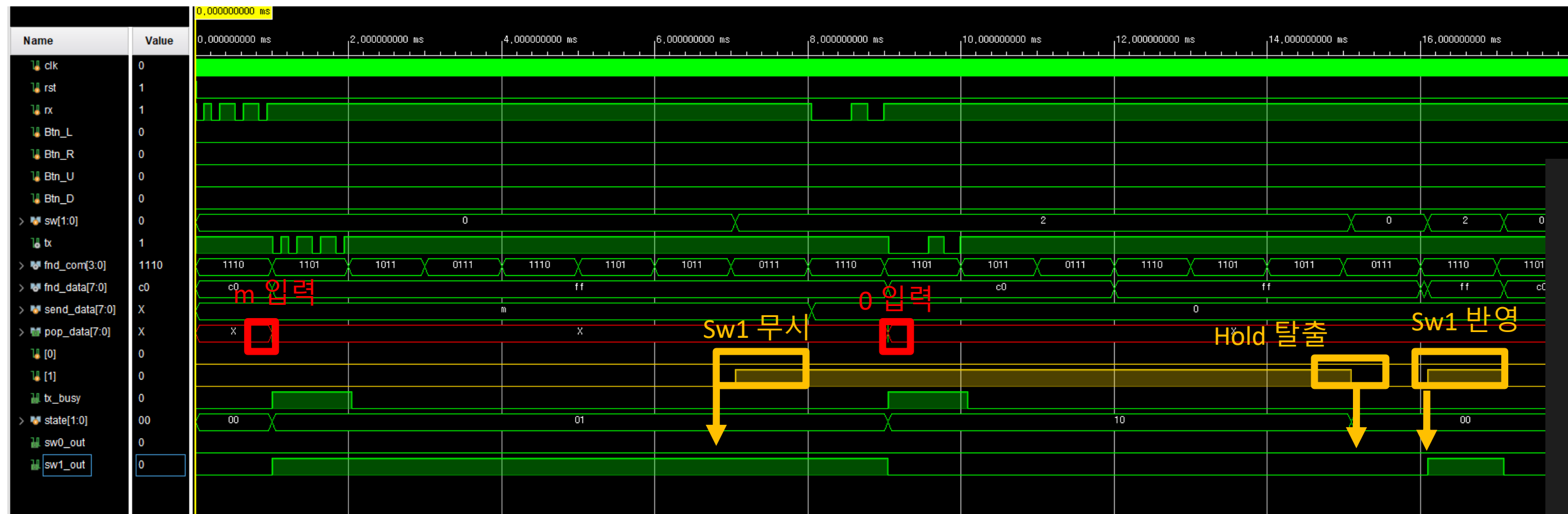
```
sw[0] = 1'b0;
#(1 * MS);
```

- Pc -> m, t 입력 -> Pc모드로 진입(sw 무시) -> n(0) 입력 -> Hold 모드 진입 -> sw[0] 입력 -> sw[0]은 hold 모드에서 그대로 반영

04

Simulation 결과

MODE_SEL : sw1



```
// PC 모드 진입 → Watch-SS:ms (mode=1, view=0)
// 보드 sw1=1 올려도 무시 확인 → '0' 수신해 HOLD로 → sw1 내리면 IDLE 복귀
```

```
send_data = "m";
send_uart(send_data);
#(5 * MS);
```

```
sw[1] = 1'b1;
#(1 * MS);
```

```
send_data = "0";
send_uart(send_data);
#(5 * MS);
```

```
sw[1] = 1'b0;
#(1 * MS);
```

```
sw[1] = 1'b1;
#(1 * MS);
```

```
sw[1] = 1'b0;
#(1 * MS);
```

- Pc -> m 입력 -> Pc모드로 진입(sw 무시) ->
sw[1] 입력 -> n(0) 입력 -> Hold 모드 진입 ->
sw[1] 입력 -> hold 모드 탈출 -> sw[1] 입력 시 반영됨

04

Simulation 결과

Run/stop/clear

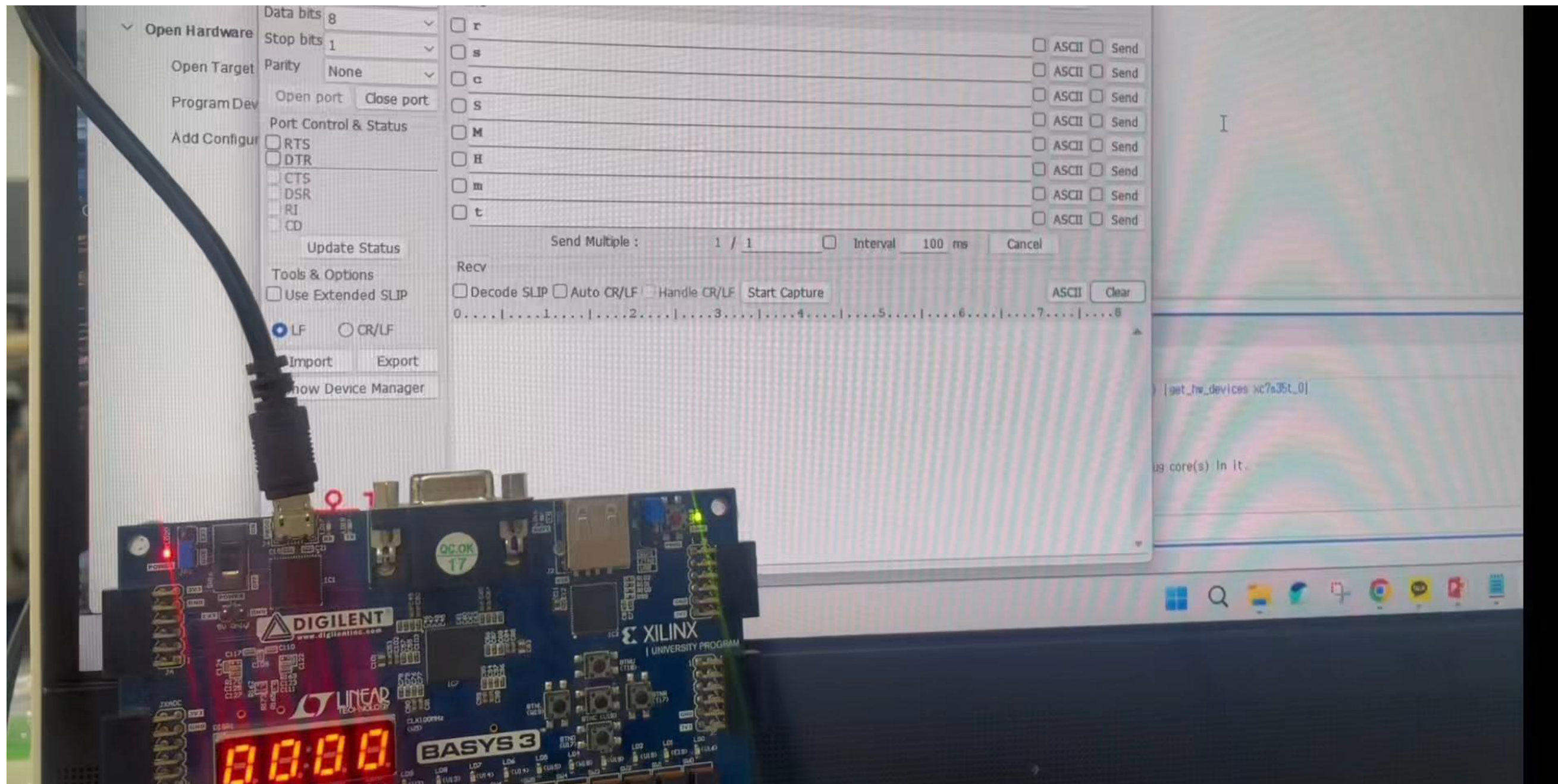


- Run 입력 -> run 입력 -> 이미 run 상태이기 때문에 변화X -> stop 입력 -> i_runstop 종료 -> clear 입력

05

동작 영상

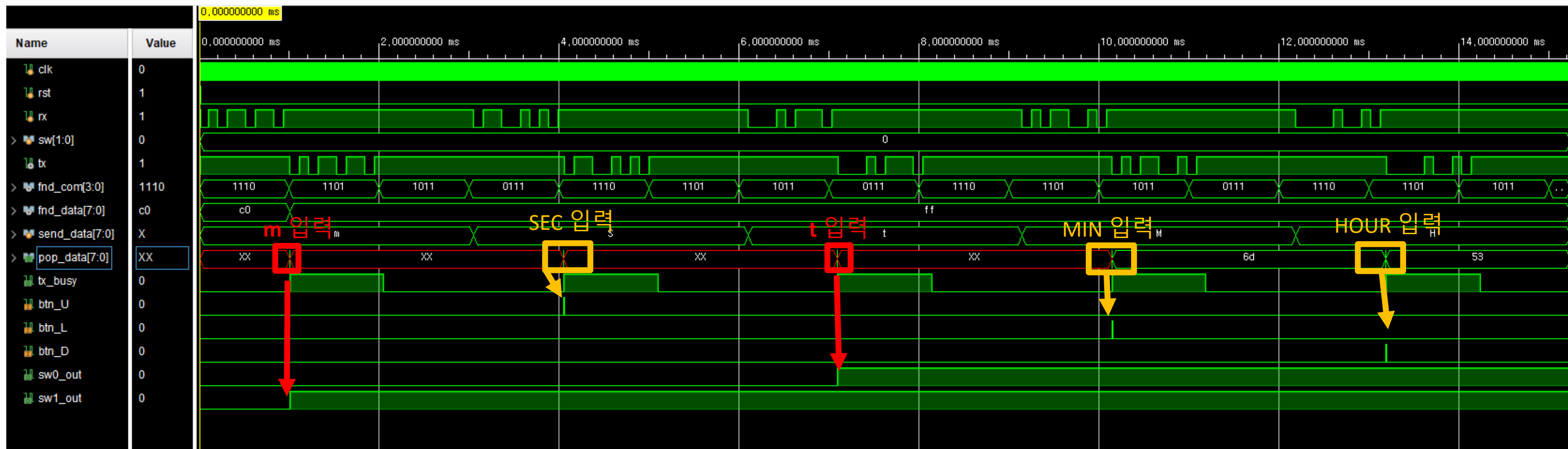
Run/stop/clear



04

Simulation 결과

Sec/Min/Hour

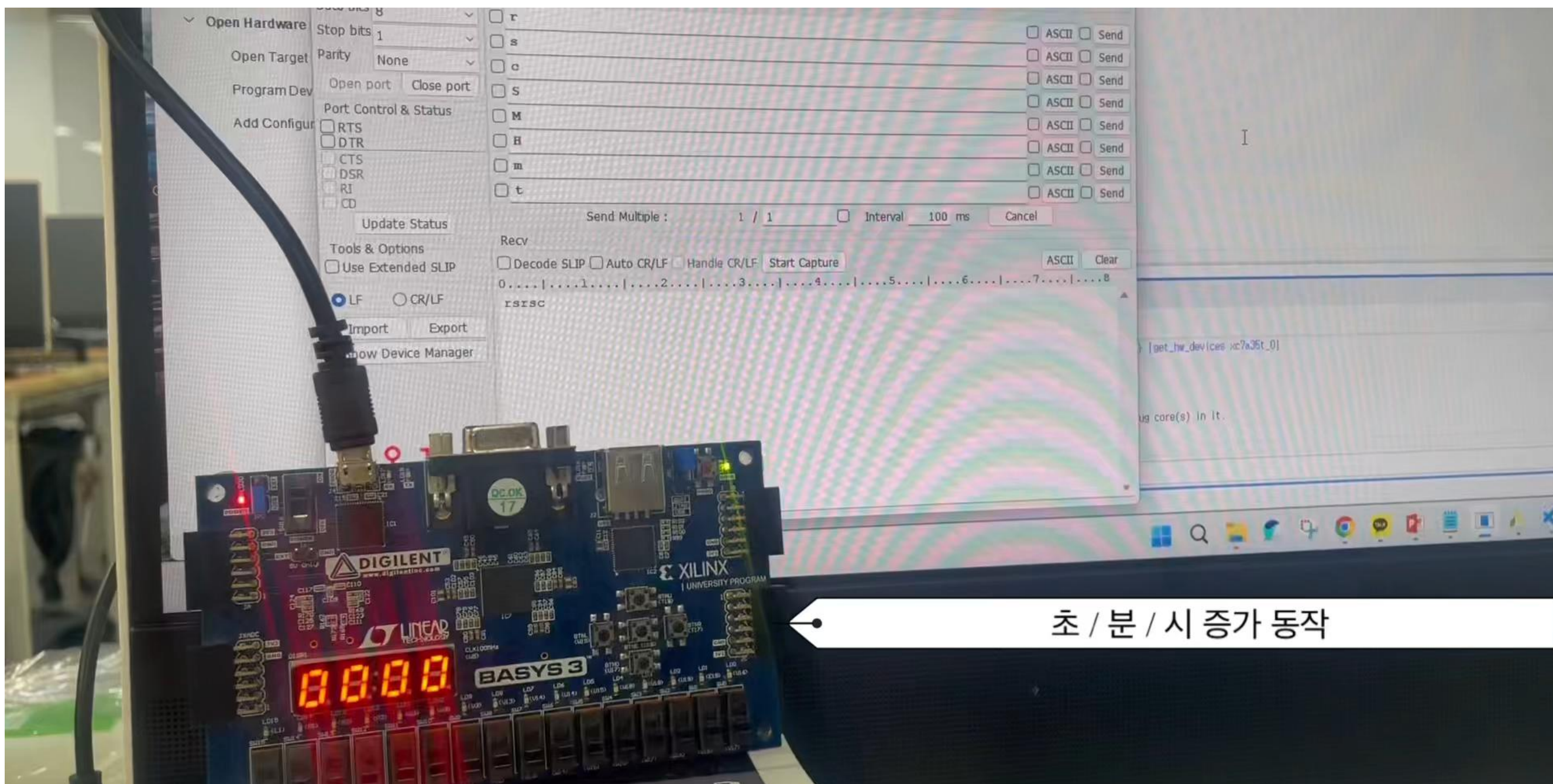


- m 입력 -> watch로 변경 -> SEC 입력 -> t 입력 ->
시 : 분 으로 변경 -> MIN 입력 -> HOUR 입력

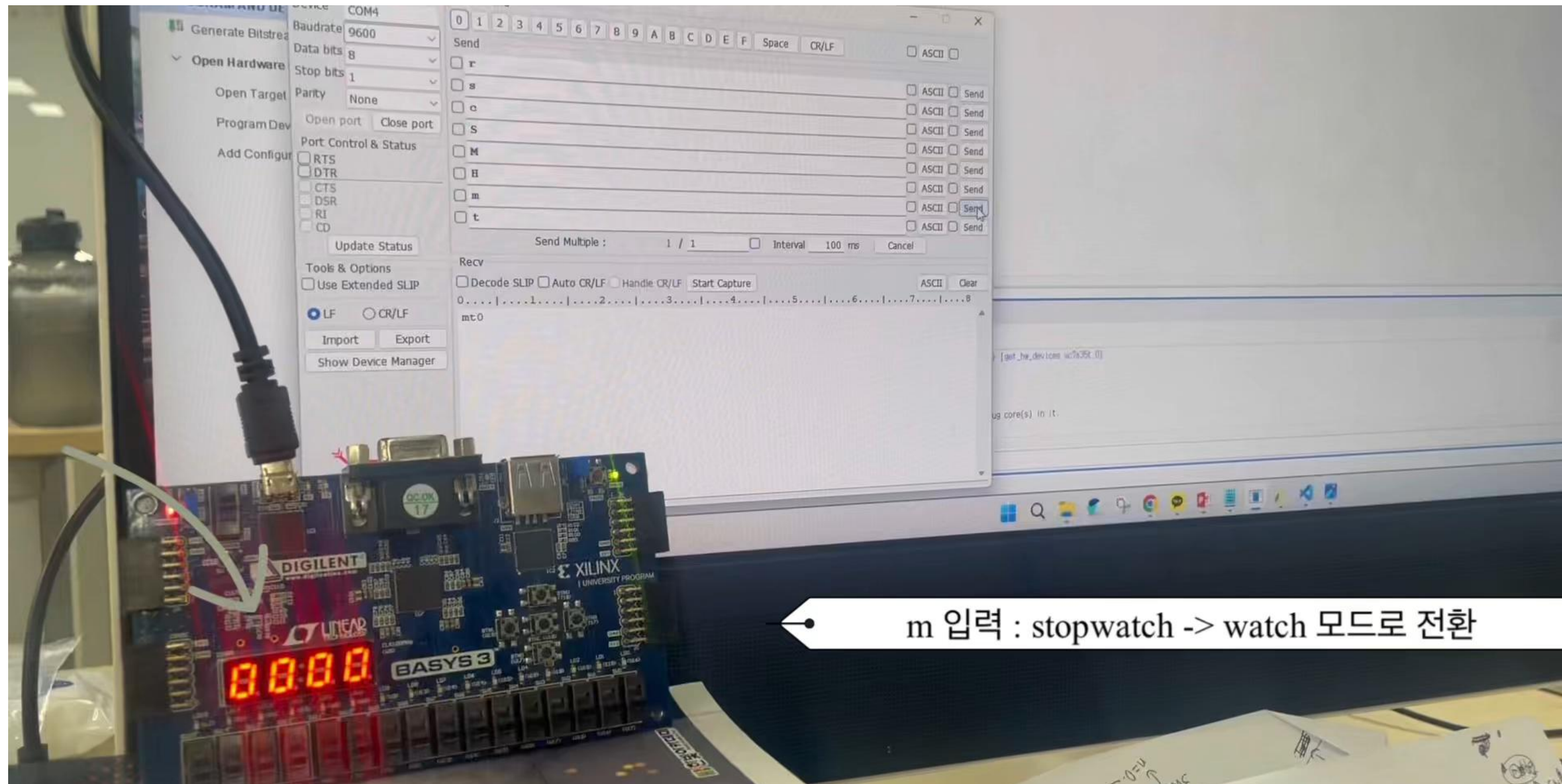
05

동작 영상

Sec/Min/Hour



05 동작 영상 Mode_sel



06

Trouble shooting

문제 발생

Mode_sel의 결과값을 2비트의
하나의 결과로 받았을때 비트
순서 및 오류 발생으로 상태가
꼬여 결과 값이 전달되지 않음



해결 방안

sw1(모드) / sw0(뷰)를 분리
출력 -> 의미·연결을 명확화

THANK YOU
